DEPI GRADUATION PROJECT | FULL STACK .NET TRACK

4.6 Additional Deliverables

MediCare — API Documentation, Testing Plan & Deployment Strategy

Team 1
Submission: 6 Nov 2026

6.1 API & Internal Service Endpoints Documentation

MediCare exposes internal MVC/JSON endpoints for interactive client components (`FullCalendar.js` and `Chart.js`), a real-time **SignalR** hub, and external notification integrations:

Endpoint / Method Signature	Layer / Component	Auth Role	Description & Payload Behavior
`GET /Doctors/Search (IDoctorRepository.Search)`	`MediCare.Web` to `MediCare.Data`	Public / Patient	Filters approved doctors (`IsApproved = true`) by `Specialty`, `ClinicAddress`, and open `AvailabilitySlots`.
`POST /Appointments/Book (BookAppointment)`	`AppointmentsController` to `AppointmentService`	`Patient`	Validates inputs, begins an atomic `IUnitOfWork` transaction, checks `HasConflict()`, saves appointment as `Pending`, and triggers `AppointmentHub.NotifyDoctor()`.
`POST /Appointments/Confirm & Reject`	`AppointmentsController` to `AppointmentService`	`Doctor`	Updates appointment status (`Confirmed` or `Rejected`), commits via `UnitOfWork`, and pushes real-time status to the patient via `AppointmentHub.NotifyPatient(status)`
`POST /Appointments/Cancel & Reschedule`	`AppointmentsController` to `AppointmentService`	`Patient`, `Doctor`	Validates cancellation policy window or new slot availability (`UC-15`), updates status to `Cancelled` or `Pending`, frees the original slot, and dispatches alerts (`UC-14`).
`WSS /appointmentHub (AppointmentHub)`	`MediCare.Web` (SignalR)	`Patient`, `Doctor`	Pushes instant browser notifications wher a new booking is created or when a booking request is confirmed or rejected.
`GET /Admin/Analytics & GET /Admin/Export`	`AdminController` to `MediCare.Services`	`Admin`	Aggregates monthly appointments, popular specialties, patient demographics and consultation revenues for **Chart.js** rendering and **PDF/Excel** export.

6.2 Testing & Validation Plan

The testing strategy validates functional workflows, concurrency constraints, security boundaries, and UI/UX usability targets:

1. Unit Testing

- Targets isolated business rules inside `MediCare.Services` using **Dependency Injection** and mocked repository/notification interfaces (`IUnitOfWork`, `INotificationSender`).
- Verifies schedule range validation (`EndTime` later than `StartTime`, `DayOfWeek` between 0 and 6), prescription constraints (`DurationDays` greater than 0), and `NotificationFactory.Create(channel)` instantiation.

2. Integration & Concurrency Testing

- Validates EF Core repositories and `UnitOfWork` transactions against SQL Server to confirm that concurrent booking requests for the same `(DoctorId, AppointmentDate, StartTime)` result in **0 double bookings** (`NFR-03`).
- Verifies Role-Based Access Control (`FR-02`) and strict record ownership checks (`FR-11`, `FR-12`, `UC-06`, `UC-11`) so patients and doctors can only access authorized records, achieving at least a **95% test suite pass rate** with zero critical open bugs (`NFR-08`).

3. User Acceptance Testing (UAT) & Prototype Validation

- Executed using **100% synthetic mock data** (`NFR-03`, `NFR-06`) across desktop and mobile browsers (`NFR-07`).
- Evaluates the 5 core tasks defined in the **UI/UX Prototype Validation Checklist** (Find a doctor, Book an appointment, Manage schedule, Review activity, and Keyboard/WCAG 2.2 AA Accessibility) to ensure at least **80% of test users** complete core workflows without guidance (`NFR-05`).

6.3 Deployment Strategy

Hosting Environment

Hosted on **Microsoft Azure App Service** backed by **Azure SQL Database** and **Azure SignalR Service**, targeting at least **99.0% availability** during evaluation windows (`NFR-04`, `NFR-09`). All credentials (DB strings, Twilio, SendGrid) are stored in protected environment secrets (`NFR-02`).

CI/CD Pipelines

All code changes are integrated via reviewed **GitHub Pull Requests** (`NFR-07`) using feature branching. Automated **GitHub Actions / Azure DevOps** pipelines trigger on merges to `main`, running `dotnet build`, unit/integration test suites, EF Core migrations, and Azure deployment.

Scaling Considerations

The **Layered MVC Monolith** structure and interface-based services (`INotificationSender`, `IGenericRepository<T>`) allow seamless switching between mock providers and cloud APIs (`NFR-10`). Offloading WebSockets to **Azure SignalR Service** and using a 3NF schema supports smooth cloud scaling.